

SYSTEM DESIGN FOR AI ENGINEERS

AI Cost Optimization System Design for AI Engineers

Design scalable LLM, RAG, and agentic AI systems that reduce token spend, control inference costs, and improve production ROI.

LLM FinOps, RAG efficiency, inference routing, caching, model selection, observability, and production cost governance



AI Cost Optimization System Design for AI Engineers

Production Systems, LLMOps, Agentic AI, FinOps, and Interview Mastery

Copyright © 2026 Lamhot Siagian / AI Engineering Insider.

All rights reserved. No part of this publication may be reproduced, distributed, or transmitted in any form or by any means without prior written permission, except for brief quotations in reviews and certain noncommercial uses permitted by copyright law.

Brand: **AI Engineering Insider**. First electronic edition, June 2026.

Version 1.0.

Disclaimer.

This book is provided for educational purposes only. The author and publisher make no warranties regarding completeness or accuracy of the content. **All pricing, model names, discount percentages, rate limits, and provider-specific behaviors change frequently.** Every concrete number in this book is illustrative. *Always verify current provider pricing and capabilities against official documentation before making production or financial decisions.* Cost figures are expressed as formulas with explicit variables wherever possible so the reasoning survives price changes. Nothing here is financial, legal, or investment advice.

Trademarks (Anthropic, Claude, OpenAI, GPT, NVIDIA, Kubernetes, and others) belong to their respective owners and are used for identification only.

Preface

Most engineering books teach you how to make AI systems *work*. This one teaches you how to make them *work profitably*. In the first wave of large language model (LLM) adoption, teams shipped impressive demos and discovered the bill at the end of the month. A single autonomous agent left in a retry loop, an unbounded conversation history, or a RAG pipeline that stuffs fifty chunks into every prompt can quietly multiply spend by ten or a hundred times without changing a single line of the user-facing feature.

Cost is now a first-class engineering constraint, on par with latency and accuracy. The engineers who get promoted, who pass senior system-design interviews, and who keep their products solvent are the ones who can answer a deceptively simple question: *where is the money going, and what is the cheapest architecture that still meets the quality bar?*

This book is a deep expansion of the *AI Cost Optimization Complete Learning Roadmap*. Every phase, term, checklist item, and provider tip in that roadmap becomes a full treatment here: what it means, why it matters, where it creates cost in production, how to implement and measure it, the mistakes engineers make, and how to discuss it in a senior interview. The structure of every concept follows one spine:

**Definition → Why it matters → Production impact → Cost impact →
Implementation pattern → Common mistakes → Interview explanation.**

You will find runnable Python, TypeScript, YAML, and SQL; compilable architecture diagrams; production case studies; and at least five senior-level interview questions in every chapter. Read it cover to cover to build a complete mental model, or jump to the chapter that matches the fire you are fighting today.

— Lamhot Siagian, *AI Engineering Insider*

Who This Book Is For

This book is written for engineers who own, or want to own, the cost line of an AI product:

- **AI & ML Engineers** building features on top of LLM APIs or open-weight models.
- **LLMOps / MLOps Engineers** responsible for serving, evaluation, and observability.
- **Agentic AI Builders** shipping tool-using and multi-agent systems where loops create runaway spend.
- **SRE / Platform Engineers** running GPU fleets, autoscaling, and multi-tenant inference.
- **Senior interview candidates** who must reason about quality–cost–latency tradeoffs on a whiteboard.
- **Technical founders and EMs** who need defensible unit economics and gross margin.

Assumed background. You can read Python, you have called an LLM API, and you understand basic ideas like embeddings, vectors, and HTTP services. You do *not* need prior FinOps experience — we build the financial reasoning from scratch.

How to Use This Book

The book follows the roadmap’s recommended learning sequence: token economics, pricing models, prompt and context optimization, caching, batching, RAG, agentic control, tool-call optimization, observability, evaluation, infrastructure, product unit economics, governance, and finally advanced routing and inference optimization.

Throughout, recurring visual signposts help you navigate:

- **Definition box** — precise meaning of a term, stripped of fluff.
- **Cost Model box** — the formula that governs the spend, with named variables.
- **Optimization Playbook** — concrete levers, ordered by leverage.
- **Common Mistake / Anti-Pattern** — the expensive errors to avoid.
- **Production Case Study** — a realistic end-to-end scenario with numbers.
- **Interview Questions** — senior-level Q&A with model answers.
- **Implementation Checklist** — a do-this list you can lift into a design doc.

You will also see short inline tags — **Senior Engineer Mental Model**, **Summary Notes**, and **Metrics to Monitor** — that compress a section into something you can repeat under pressure.

Treat all dollar amounts, token rates, and discount percentages as *placeholders for the reasoning*, not as current quotes. The relationships (output costs more than input, batch is cheaper than real-time, caching reuses static prefixes) are stable; the exact multipliers are not. Where a calculation matters, the formula is given so you can plug in today’s verified prices.

The AI Cost Optimization Mental Model

Every AI system, no matter how complex, decomposes into a small set of cost-producing activities. If you can name them and attach a unit to each, you can optimize any system.

Total AI Cost = LLM inference cost + embedding cost + vector DB cost
+ reranking cost + tool/API cost + storage cost
+ orchestration cost + evaluation cost
+ logging/observability cost + human review cost
+ infrastructure cost

That additive stack is the *accounting* view — it tells you which line items exist. The *action* view is the simplified master formula, which tells you what to actually *do*:

AI Cost = $\underbrace{\text{Right Model}}_{\text{route}}$ + $\underbrace{\text{Fewer Tokens}}_{\text{compress}}$ + $\underbrace{\text{Cache Reuse}}_{\text{reuse}}$ + $\underbrace{\text{Batch Offline}}_{\text{defer}}$ + $\underbrace{\text{Controlled Loops}}_{\text{bound}}$ + $\underbrace{\text{Monitor}}_{\text{observe}}$

Senior Engineer Mental Model. Cost optimization is not one trick; it is six independent levers applied in order of leverage. Routing to a smaller model and reusing cached prefixes typically beat every micro-optimization downstream. Always ask: which lever moves the biggest line item in my stack?

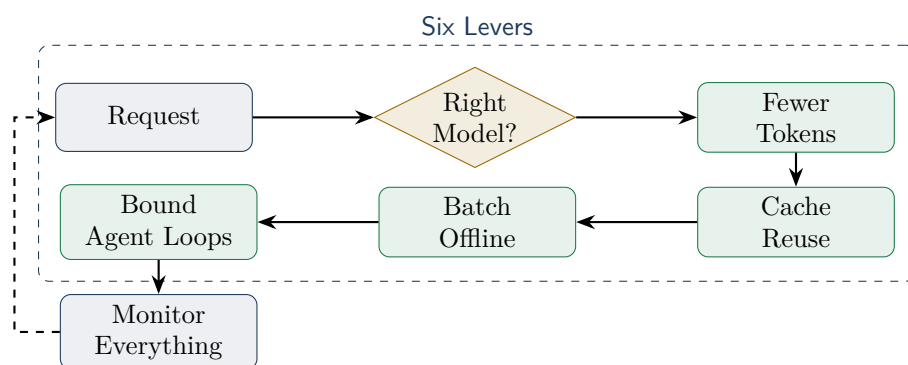


Figure 1: The six-lever loop: route, compress, reuse, defer, bound, observe — then feed measurements back into routing decisions.

The unit-metric layer. Totals hide problems; *units* expose them. The roadmap's key unit metrics convert an opaque monthly bill into per-event numbers you can compare against revenue:

Unit metric	Question it answers
cost / request	What does one API-shaped interaction cost?
cost / user	What does an active user cost per month?
cost / task	What does one <i>successful</i> unit of work cost?
cost / agent-trajectory	What does one full agent run cost end to end?
cost / eval-run	What does one evaluation pass cost?
cost / document	What does processing one document cost?
accuracy-per-dollar	How much quality do we buy per dollar?
tokens-per-task	How token-efficient is our workflow?
tool-calls-per-task	Are agents over-calling tools?
retrieval-cost-per-answer	What does RAG add per answer?
human-review-cost-per-decision	What does the human-in-the-loop cost?

Summary Notes. *“I never optimize a total. I optimize a unit metric tied to revenue — cost per successful task — because that is the number that decides whether the product has a margin.”*

Roadmap-to-Chapter Coverage Map

The table below maps every phase and cluster of the source roadmap to the chapters that expand it, so you can confirm no term is dropped.

Roadmap phase	Roadmap cluster	Chapter(s)
Foundations	Total AI Cost stack, unit metrics	Ch. 1
Phase 1	Token Economics	Ch. 2
Phase 1	Pricing Model Anatomy	Ch. 3
Phase 1	Business Unit Metrics	Ch. 4
Phase 2	Model Selection Strategy	Ch. 5
Phase 2	LLM Routing Patterns	Ch. 6
Phase 2	RAG Cost Optimization	Ch. 7
Phase 3	Prompt Engineering for Cost	Ch. 8
Phase 3	Context Window Management	Ch. 9
Phase 3	Caching Strategies	Ch. 10
Phase 4	Agent Loop Budget	Ch. 11
Phase 4	Multi-Agent Cost	Ch. 12
Phase 4	Tool-Call Optimization	Ch. 13
Phase 4	Agent Memory Cost	Ch. 14
Phase 5	Inference Engineering	Ch. 15
Phase 5	Serving Platforms	Ch. 16
Phase 5	Observability Tools & FinOps	Ch. 17
Phase 5	Batch & Eval Strategy	Ch. 18
Provider tips	Claude API Token Efficiency	Ch. 19
Provider tips	OpenAI API Token Efficiency	Ch. 20
Advanced	Advanced Topics (MoE, multimodal, governance)	Ch. 21
Projects	7 Practical Projects	Ch. 22
Mastery	Mastery Checklist & Interview Review	Ch. 23
Reference	Formulas, templates, glossary, index	Appendices

A complete alphabetical **Glossary** and an **Index-style term list** at the back of the book let you confirm coverage of individual terms.

Contents

Preface	iii
Who This Book Is For	iv
How to Use This Book	v
The AI Cost Optimization Mental Model	vi
Roadmap-to-Chapter Coverage Map	viii
1 AI Cost Optimization Fundamentals	1
1.1 Why an AI cost discipline exists at all	1
1.2 The Total AI Cost stack, line item by line item	1
1.2.1 LLM inference cost ($C_{\text{inference}}$)	1
1.2.2 Embedding cost ($C_{\text{embedding}}$)	2
1.2.3 Vector DB cost (C_{vectordb})	2
1.2.4 Reranking cost (C_{rerank})	2
1.2.5 Tool / API cost (C_{tool})	2
1.2.6 Storage and file cost (C_{storage})	2
1.2.7 Orchestration cost ($C_{\text{orchestration}}$)	2
1.2.8 Evaluation cost (C_{eval})	2
1.2.9 Logging / observability cost ($C_{\text{observability}}$)	2
1.2.10 Human review cost (C_{human})	3
1.2.11 Infrastructure cost (C_{infra})	3
1.3 From totals to units: the metrics that drive decisions	3
1.4 A worked example: cost per request and monthly COGS	3
1.5 Production case study	5
1.6 Optimization playbook and checklist	6
1.7 Common mistakes	6
1.8 Interview questions	6
Q1. Walk me through the full cost stack of an AI feature.	6
Q2. Why is cost per successful task better than cost per request?	6
Q3. A monthly bill doubled with flat user count. How do you diagnose it?	6
Q4. How would you set unit-economics targets for a new AI feature?	7
Q5. When is the non-inference part of the stack the thing to optimize?	7
2 Token Economics	8
2.1 Tokens are the currency of AI systems	8
2.2 Input vs. output tokens, and why output costs more	8
2.3 Cached input tokens and long-context tiers	8
2.4 The hidden tokens: reasoning, schemas, history, and JSON	9
2.5 Multi-turn accumulation and the token-flow diagram	9
2.6 Streaming, temperature, retries, and other cost modifiers	9
2.7 A token-aware cost estimator	10
2.8 Production case study	11

2.9	Interview questions	12
	Q1. Why do output tokens cost more than input tokens?	12
	Q2. A multi-turn chatbot's cost grows through a session. Why, and what do you do?	12
	Q3. What are "hidden" tokens and how do you control them?	12
	Q4. How does caching change the token-cost equation?	13
	Q5. How do retries and fallbacks affect unit cost, and how do you bound them? .	13
3	Pricing Model Anatomy	14
3.1	The pricing decision sits above every optimization	14
3.2	The pay-per-token family	14
3.3	The add-on meters: tools, storage, sessions, search	14
3.4	Self-hosting and the CapEx/OpEx tradeoff	15
3.5	Pricing structures you sell, and ones you must defend against	15
3.6	Comparing API vs. self-hosted monthly cost	15
3.7	Production case study	16
3.8	Interview questions	17
	Q1. When does self-hosting beat a pay-per-token API?	17
	Q2. What is the Batch API tradeoff and when do you use it?	17
	Q3. Per-seat pricing looks simple — what is the hidden risk?	17
	Q4. How do you defend a free tier against cost abuse?	17
	Q5. Design a pricing strategy for a workload that is half batch, half interactive. .	17
4	Business Unit Metrics for AI Products	18
4.1	Turning engineering cost into product economics	18
4.2	The unit-metric ladder	18
4.3	The quality–cost–latency triangle	18
4.4	Accuracy-per-dollar, LTV, and margin-aware features	19
4.5	Gross-margin simulation	19
4.6	Production case study	20
4.7	Interview questions	21
	Q1. Explain the quality–cost–latency triangle with an example.	21
	Q2. Why prefer accuracy-per-dollar over raw accuracy when choosing a model? .	21
	Q3. A \$20/seat product has a few users costing \$60/month in AI. What do you do?	21
	Q4. How do you set a cost-per-task target for a new feature?	21
	Q5. How does LTV constrain AI feature design?	21
5	Model Selection Strategy	22
5.1	The highest-leverage decision in the stack	22
5.2	The selection axes	22
5.3	Cheap-first architectures and escalation	22
5.4	A model selection policy	23
5.5	Production case study	24
5.6	Interview questions	25
	Q1. How do you choose a model for a new feature?	25
	Q2. Explain cheap-first with escalation and its expected cost.	25
	Q3. When is fine-tuning the cost-optimal choice over prompting?	25
	Q4. What makes a good escalation/confidence signal?	25
	Q5. How do distilled models change the cost calculus?	25

6	LLM Routing Patterns	26
6.1	From selection to automatic routing	26
6.2	Routing strategies	26
6.3	A FastAPI complexity router	26
6.4	Production case study	28
6.5	Interview questions	29
	Q1. Rule-based vs. semantic vs. learned routing — tradeoffs?	29
	Q2. Which model should do the routing, and why?	29
	Q3. How do you make routing resilient to provider failure?	29
	Q4. How do you prevent a router from quietly hurting quality?	29
	Q5. What is the cost of multi-model ensembles and when is it justified?	29
7	RAG Cost Optimization	30
7.1	RAG is a cost system, not just a quality system	30
7.2	The retrieval pipeline and its meters	30
7.3	Vector DB internals and index economics	30
7.4	RAG vs. long-context, and pre-computation	31
7.5	Benchmarking RAG cost vs. quality	31
7.6	Production case study	32
7.7	Interview questions	33
	Q1. Where is the dominant cost in a RAG query, and how do you cut it?	33
	Q2. Does a reranker increase or decrease cost?	33
	Q3. RAG vs. long-context — how do you choose?	33
	Q4. HNSW vs. IVF, and hot vs. cold indexes — cost implications?	33
	Q5. How do you keep indexing cost down as the corpus grows?	33
8	Prompt Engineering for Cost	34
8.1	Prompts are recurring cost, not one-time text	34
8.2	Input-side levers	34
8.3	Output-side levers	34
8.4	Prompts as managed artifacts	34
8.5	Prompt template optimization in code	35
8.6	Production case study	36
8.7	Interview questions	36
	Q1. Why is the system prompt a cost concern at scale?	36
	Q2. How do structured outputs reduce cost?	36
	Q3. What is prompt compression and when is it worth it?	37
	Q4. Dynamic vs. static few-shot — cost and quality tradeoff?	37
	Q5. How do you catch a prompt-driven cost regression?	37
9	Context Window Management	38
9.1	Context is a budget you spend every turn	38
9.2	Memory architectures	38
9.3	Pruning, ranking, and the lost-in-the-middle problem	38
9.4	Static vs. dynamic separation and map-reduce	39
9.5	Context pruning and ranking in code	39
9.6	Production case study	40
9.7	Interview questions	40
	Q1. Why can adding more context lower quality and raise cost simultaneously?	40
	Q2. How do you bound the cost of a long multi-turn session?	41
	Q3. What is static/dynamic separation and why does it matter?	41
	Q4. When do you use map-reduce over a document?	41

Q5. Recency vs. relevance — how do you decide what to keep?	41
10 Caching Strategies	42
10.1 Reuse is the cheapest token	42
10.2 The cache hierarchy	42
10.3 Operating a cache: hit rate, TTL, invalidation	42
10.4 Security isolation in cache	43
10.5 A Redis semantic + exact cache	43
10.6 Production case study	44
10.7 Interview questions	45
Q1. Walk through the cache hierarchy and the order you check it.	45
Q2. What breaks provider prompt caching?	45
Q3. How do you cache safely in a multi-tenant system?	45
Q4. Semantic vs. exact caching — tradeoffs?	45
Q5. How do you decide TTL and invalidation policy?	45
11 Agentic Workflow Cost Control	46
11.1 Agents turn cost from linear into open-ended	46
11.2 The dimensions of an agent budget	46
11.3 An agent budget guardrail	46
11.4 Production case study	48
11.5 Interview questions	49
Q1. Why are agents a distinct cost-control problem?	49
Q2. What budget dimensions do you enforce, and why all of them?	49
Q3. How do early stopping and confidence-based escalation save money?	49
Q4. Design a kill switch for production agents.	49
Q5. How do you set the budget numbers themselves?	49
12 Multi-Agent Cost Optimization	50
12.1 Coordination multiplies cost	50
12.2 Topologies and their cost	50
12.3 Limiting fan-out in code	50
12.4 Production case study	51
12.5 Interview questions	52
Q1. Why can multi-agent systems explode in cost?	52
Q2. Plan-then-execute vs. debate — cost and quality tradeoff?	52
Q3. How do you bound a multi-agent system’s total cost?	52
Q4. What is inter-agent message cost and how do you reduce it?	52
Q5. How does sub-agent model routing help?	53
13 Tool-Call Optimization	54
13.1 Tools have two costs: schema and execution	54
13.2 Cutting schema overhead	54
13.3 Cutting execution cost	54
13.4 Dynamic tool registration	54
13.5 Production case study	55
13.6 Interview questions	56
Q1. What are the two costs of tool use?	56
Q2. How do you reduce tool-schema overhead?	56
Q3. Which tool results can you cache, and what is the risk?	56
Q4. Does parallel tool execution reduce cost?	57
Q5. How do you bound the cost of unreliable tools?	57

14 Agent Memory Cost	58
14.1 Memory is a buy-vs-rent decision	58
14.2 Context vs. retrieval, and summarization	58
14.3 Human-in-the-loop saves and reflection overuse	58
14.4 Summarizing and storing agent memory	58
14.5 Production case study	59
14.6 Interview questions	60
Q1. Frame agent memory as a cost decision.	60
Q2. When is summarization worth its own cost?	60
Q3. What is reflection overuse and how do you bound it?	60
Q4. How do approval checkpoints affect cost?	61
Q5. How does a trajectory cache save money?	61
15 Inference Engineering	62
15.1 When you own the GPU, you own the efficiency	62
15.2 Quantization and model compression	62
15.3 Scheduling: batching and speculative decoding	62
15.4 Memory and kernels: KV-cache, PagedAttention, Flash Attention	63
15.5 A batching scheduler sketch	63
15.6 Production case study	64
15.7 Interview questions	65
Q1. What single technique most improves self-hosted throughput, and why?	65
Q2. How does quantization lower cost, and what is the risk?	65
Q3. Explain speculative decoding's economics.	65
Q4. What is PagedAttention and why does it matter for cost?	65
Q5. Tensor vs. pipeline parallelism — when and at what cost?	65
16 Serving Platforms	66
16.1 The platform decides your utilization	66
16.2 The inference servers	66
16.3 Scaling and purchasing levers	66
16.4 A GPU inference deployment	66
16.5 Production case study	68
16.6 Interview questions	69
Q1. What is the dominant cost lever in self-hosted serving?	69
Q2. How do you blend spot and reserved instances?	69
Q3. What signal should drive GPU autoscaling, and why not CPU?	69
Q4. How do warm pools and cold starts trade off?	69
Q5. What does multi-tenant inference buy and what does it require?	69
17 Observability and FinOps	70
17.1 You cannot optimize what you cannot see	70
17.2 The tooling landscape	70
17.3 What to measure and govern	70
17.4 OpenTelemetry-style cost tracing	70
17.5 Production case study	72
17.6 Interview questions	73
Q1. What attributes must an LLM cost span carry?	73
Q2. How do you keep observability from becoming a cost problem itself?	73
Q3. Design an anomaly alert for AI spend.	73
Q4. How does prompt-version tracking aid cost control?	73
Q5. What is a budget guardrail and how is it enforced?	73

18 Batch and Evaluation Strategy	74
18.1 Two cheap habits: defer what you can, measure what you ship	74
18.2 Batching: defer for the discount	74
18.3 Evaluation cadence and its cost	74
18.4 Eval cost tracking	75
18.5 Production case study	76
18.6 Interview questions	76
Q1. When should work go to the Batch API and what does it cost you?	76
Q2. Why does idempotency matter in batch jobs?	77
Q3. Design an eval cadence that balances assurance and cost.	77
Q4. How do you reduce LLM-as-judge cost?	77
Q5. What is a CI token-cost gate and why use it?	77
19 Claude API Token Efficiency	78
19.1 Provider-specific levers on top of the general principles	78
19.2 Anthropic prompt caching	78
19.3 Output token control	79
19.4 Context and tier strategy	79
Q1. How does Anthropic prompt caching work and how do you maximize hits?	81
Q2. Why use Haiku as a router?	81
Q3. How do you control output cost on Claude?	81
Q4. How do batch and caching combine on Claude?	81
Q5. What breaks Claude prompt caching and how do you prevent it?	81
20 OpenAI API Token Efficiency	83
20.1 The same levers, a different caching contract	83
20.2 OpenAI prompt caching (automatic prefix caching)	83
20.3 Model tier and reasoning strategy	84
20.4 Batch, embeddings, output, and context	84
Q1. How does OpenAI prompt caching differ from Anthropic's, and how do you exploit it?	86
Q2. Why set <code>max_completion_tokens</code> lower for reasoning models?	86
Q3. How do strict JSON schema outputs reduce cost?	86
Q4. When and how do you use the Batch API, and does it stack with caching?	86
Q5. What invalidates OpenAI prefix caching, and how do you keep prefixes stable?	86
21 Advanced AI Cost Optimization Topics	88
21.1 The frontier of cost engineering	88
Q1. Why are MoE models attractive for inference cost?	91
Q2. How do Matryoshka and binary embeddings cut cost?	91
Q3. What is the single biggest lever for multimodal cost?	91
Q4. Explain SLA-aware model selection.	91
Q5. How do prompt-injection cost attacks work and how do you defend?	91
22 Practical Projects	92
22.1 From concepts to a portfolio	92
22.2 Project 1 — LLM Cost Calculator	92
22.3 Project 2 — Model Router	93
22.4 Project 3 — RAG Cost Optimizer	93
22.5 Project 4 — Agent Budget Controller	94
22.6 Project 5 — LLM Observability Dashboard	95
22.7 Project 6 — Prompt Cache Experiment	95

22.8 Project 7 — Eval Cost Pipeline	96
Q1. Which project would you build first and why?	97
Q2. How does the router project prove it saved money?	97
Q3. What makes the RAG optimizer credible?	97
Q4. How do you test an agent budget controller?	97
Q5. What is the highest-signal metric across all seven projects?	97
23 Mastery Checklist and Interview Review	98
23.1 You have mastered AI cost optimization when...	98
23.2 Production diagnostic checklist	99
23.3 Architecture review checklist	99
23.4 Cost review & FinOps governance checklist	99
23.5 Final mock interview — the capstone system design	99
Q1. Where does cost concentrate at 1M MAU, and what is your first move? . . .	101
Q2. How do you guarantee the platform cannot blow its budget?	101
Q3. A tenant's cost suddenly triples. Diagnose and contain.	101
Q4. How do you keep quality while cutting cost aggressively?	101
Q5. What single metric do you commit to in the design review?	102
A Formula Reference	103
B Reusable Templates	104
B.1 Cost calculator template	104
B.2 Router template	104
B.3 Agent budget template	104
B.4 RAG optimization template	105
B.5 Observability dashboard template	105
B.6 Eval pipeline template	105
C Glossary	106
D Index-Style Term List	108

AI Cost Optimization Fundamentals

1.1 Why an AI cost discipline exists at all

A traditional web service has a cost curve that engineers understand intuitively: CPU, memory, bandwidth, and storage scale roughly with traffic, and a request that does “more work” costs more in a way that maps to familiar resources. Generative AI breaks that intuition. The dominant cost of an LLM feature is *tokens consumed by a model you do not host*, and that cost is only loosely correlated with user-visible effort. A one-line user question can trigger a fifty-thousand-token prompt because the system stuffed an entire document, a long chat history, and a dozen tool schemas into the context. An agent can spend dollars on a task the user thought was free because it looped eight times, retried failed tool calls, and reflected on its own output twice.

This is why AI cost optimization is its own discipline. The spend is created in places that do not appear in the product spec, it compounds across multi-turn and multi-agent flows, and it is invisible until the invoice arrives unless you instrument for it. The goal of this chapter is to give you the complete *cost surface* of an AI system — every place money is created — and the *unit metrics* that turn an opaque total into decisions.

Definition: AI cost optimization

The engineering practice of minimizing the total cost of delivering a target quality of AI output, subject to latency and reliability constraints, by controlling model choice, token volume, reuse, scheduling, agent behavior, and infrastructure — and by measuring all of it per unit of useful work.

1.2 The Total AI Cost stack, line item by line item

The master accounting identity from the front matter is worth restating, because the rest of the book is an expansion of its eleven terms.

$$\begin{aligned} \text{Total AI Cost} = & C_{\text{inference}} + C_{\text{embedding}} + C_{\text{vectordb}} + C_{\text{rerank}} + C_{\text{tool}} + C_{\text{storage}} \\ & + C_{\text{orchestration}} + C_{\text{eval}} + C_{\text{observability}} + C_{\text{human}} + C_{\text{infra}} \end{aligned}$$

1.2.1 LLM inference cost ($C_{\text{inference}}$)

The dominant line item for most API-based products. For a pay-per-token model it is

$$C_{\text{inference}} = N_{\text{in}} p_{\text{in}} + N_{\text{out}} p_{\text{out}} + N_{\text{cached}} p_{\text{cached}},$$

where N are token counts and p are per-token prices. **Why it matters:** output tokens are typically several times more expensive than input, and unbounded prompts inflate N_{in} . **Production impact:** every architectural decision — RAG, memory, agents, routing — ultimately shows up here. **Implementation:** log token counts on every call; never estimate inference cost from request *count* alone.

1.2.2 Embedding cost ($C_{\text{embedding}}$)

The cost of converting text (queries and documents) into vectors. Usually a low per-token price, but it scales with corpus size during indexing and with query volume at serve time: $C_{\text{embedding}} = (N_{\text{index}} + N_{\text{query}}) p_{\text{embed}}$. **Common trap:** re-embedding an entire corpus on every deploy instead of embedding only deltas.

1.2.3 Vector DB cost (C_{vectordb})

Storage and query cost of the vector index (e.g., HNSW or IVF structures), priced by vector count, dimensionality, replicas, and queries per second. Managed vector databases often bill on provisioned pods or read/write units, so an over-replicated “hot” index for a low-traffic feature is pure waste.

1.2.4 Reranking cost (C_{rerank})

A second-stage model (cross-encoder or LLM) that re-scores retrieved candidates. It improves precision but adds a per-query model call: $C_{\text{rerank}} = Q \cdot k \cdot p_{\text{rerank}}$ for Q queries each reranking k candidates. Often worth it; often applied to far more candidates than necessary.

1.2.5 Tool / API cost (C_{tool})

External calls an agent makes: web search, code execution, browser automation, third-party SaaS APIs. These are billed per call by the provider and can dwarf token cost in tool-heavy agents. **Production impact:** a search-augmented agent’s bill is often dominated by search API calls, not the LLM.

1.2.6 Storage and file cost (C_{storage})

Object storage for documents, uploaded files, cached artifacts, logs, and (with some providers) server-side conversation or response storage. Individually cheap, collectively non-trivial at scale, and easy to forget to garbage-collect.

1.2.7 Orchestration cost ($C_{\text{orchestration}}$)

The compute running your application logic: the API servers, queues, workflow engines, and glue that coordinate calls. Modest per request but always-on, so it dominates at very low traffic and disappears into the noise at high traffic.

1.2.8 Evaluation cost (C_{eval})

Running test suites, LLM-as-judge scoring, and regression benchmarks. It is a real, recurring spend that scales with eval cadence and dataset size — and a place teams either under-invest (no regression safety net) or over-spend (frontier judge on every PR).

1.2.9 Logging / observability cost ($C_{\text{observability}}$)

Trace ingestion, metric storage, and dashboards (LangSmith, Langfuse, Datadog, etc.). Necessary to do everything else in this book, but high-cardinality token-level logging can itself become a line item worth capping.

1.2.10 Human review cost (C_{human})

The fully loaded cost of humans in the loop: labeling, approvals, escalations, and QA. Per decision this is often the *most* expensive token in the system, which is why automating or reducing human review is a legitimate optimization with large leverage.

1.2.11 Infrastructure cost (C_{infra})

For self-hosted models: GPUs, networking, and the platform around them (Kubernetes, load balancers, autoscalers). This is CapEx-like spend that only pays off above a volume threshold — the central tension of Chapter 3.

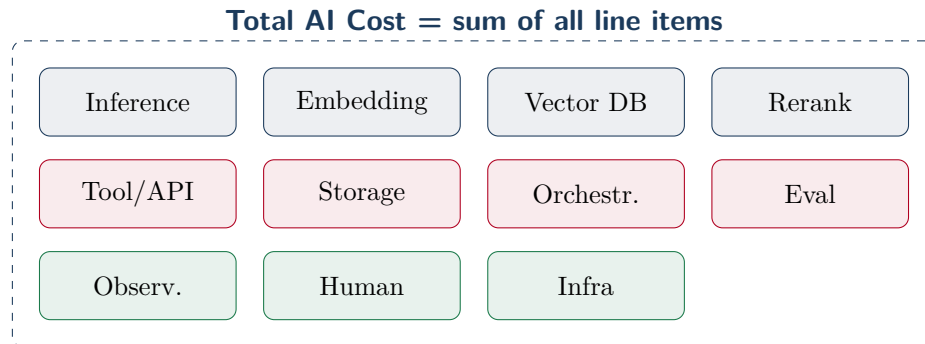


Figure 1.1: The full AI cost stack. Blue = model/data plane, red = per-call externalities, green = always-on operational cost. Optimize the biggest box first.

1.3 From totals to units: the metrics that drive decisions

A monthly total of “\$47,000” is not actionable. The same total becomes a decision when divided by the right denominator. The roadmap names eleven unit metrics; the most important distinction among them is *cost per request* versus *cost per successful task*.

Definition: Cost per successful task

Total cost attributable to a workflow divided by the number of tasks that actually succeeded (passed the quality bar), including the cost of the failed and retried attempts. This is the number that determines product margin — a system can have a cheap cost per *request* and a ruinous cost per *successful task* if it retries constantly.

$$\begin{aligned} \text{cost/request} &= \frac{C_{\text{total}}}{R}, & \text{cost/task} &= \frac{C_{\text{total}}}{T_{\text{success}}}, & \text{accuracy-per-dollar} &= \frac{\text{quality}}{C_{\text{total}}} \\ \text{tokens/task} &= \frac{N_{\text{in}} + N_{\text{out}}}{T}, & \text{tool-calls/task} &= \frac{\# \text{tool calls}}{T}, & \text{retrieval-cost/answer} &= \frac{C_{\text{embed}} + C_{\text{vectordb}} + C_{\text{rerank}}}{A} \end{aligned}$$

Metrics to Monitor. Track cost/successful-task and accuracy-per-dollar on a dashboard next to each other. The first tells you if you are solvent; the second tells you if a cheaper model would keep you solvent without hurting users.

1.4 A worked example: cost per request and monthly COGS

Listing 1.1: Cost-per-request and monthly AI COGS estimator.

```

1  """
2  Purpose: Compute cost per request and projected monthly AI Cost of Goods Sold (COGS)
3           from token usage and per-line-item rates. All prices are PLACEHOLDERS.
4  # Production note: replace these illustrative rates with current provider pricing
5           ↪ before deployment.
6  """
7
8  from dataclasses import dataclass, field
9
10 @dataclass
11 class Rates:
12     price_in: float          # $ per input token
13     price_out: float         # $ per output token
14     price_cached: float      # $ per cached input token
15     price_embed: float       # $ per embedding token
16     price_rerank_call: float = 0.0 # $ per rerank call
17     price_tool_call: float = 0.0  # $ per external tool/search call
18
19 @dataclass
20 class Usage:
21     tokens_in: int = 0
22     tokens_out: int = 0
23     tokens_cached: int = 0
24     tokens_embed: int = 0
25     rerank_calls: int = 0
26     tool_calls: int = 0
27
28 def request_cost(u: Usage, r: Rates) -> dict:
29     inference = u.tokens_in * r.price_in + u.tokens_out * r.price_out \
30         + u.tokens_cached * r.price_cached
31     embedding = u.tokens_embed * r.price_embed
32     rerank = u.rerank_calls * r.price_rerank_call
33     tools = u.tool_calls * r.price_tool_call
34     total = inference + embedding + rerank + tools
35     return {"inference": inference, "embedding": embedding,
36           "rerank": rerank, "tools": tools, "total": total}
37
38 def monthly_cogs(cost_per_request: float, requests_per_day: int,
39                 success_rate: float = 1.0, days: int = 30) -> dict:
40     monthly_requests = requests_per_day * days
41     gross = cost_per_request * monthly_requests
42     successful = monthly_requests * success_rate
43     cost_per_success = gross / successful if successful else float("inf")
44     return {"monthly_requests": monthly_requests, "monthly_cost": gross,
45           "cost_per_successful_task": cost_per_success}
46
47 if __name__ == "__main__":
48     rates = Rates(price_in=3e-6, price_out=15e-6, price_cached=0.3e-6,
49                   price_embed=0.02e-6, price_rerank_call=1e-3, price_tool_call=5e-3)
50     usage = Usage(tokens_in=4000, tokens_out=600, tokens_cached=3000,
51                   tokens_embed=800, rerank_calls=1, tool_calls=2)
52     rc = request_cost(usage, rates)
53     print("Per-request breakdown:", {k: round(v, 6) for k, v in rc.items()})
54     print("Monthly:", monthly_cogs(rc["total"], requests_per_day=20000, success_rate
55     ↪ =0.9))

```

Explanation. The estimator separates the cost stack into named line items so you can see *which* term dominates a single request, then projects it to a monthly COGS figure and — crucially — divides by the success rate to expose cost per *successful* task. **Production notes:** feed real token counts from API usage fields, not estimates; store per-request rows so you can group by feature and model later. **Failure cases:** a 90% success rate quietly raises true unit cost by $\approx 11\%$; cached-token accounting is wrong if you assume a cache hit that did not occur. **Cost implications:** this single function is the seed of the observability dashboard in Chapter 17 and Project P1 in Chapter 22.

1.5 Production case study

Case Study: Cost dashboard for a SaaS AI assistant

Context: A B2B SaaS product embeds an AI assistant; finance sees a fast-rising, unexplained model bill and asks engineering “why.”

Problem: Spend is reported only as one monthly provider invoice. There is no per-feature, per-user, or per-model attribution, so no one can say which feature is unprofitable.

Architecture: A thin logging middleware wraps every LLM/embedding/tool call, emitting a structured event `{user_id, feature, model, tokens_in, tokens_out, cached, tool_calls, success}`. Events stream to a warehouse; a scheduled job computes the unit metrics; a dashboard slices cost by feature, model, and user cohort.

Cost Drivers: Inference dominates; one “summarize entire workspace” feature sends 30k-token prompts and accounts for a disproportionate share.

Optimization Strategy: Attribute first, then act — route the cheap features to a small model, cap the expensive feature’s context, and cache the static workspace prefix.

Expected Result: With attribution alone the team usually finds that 10–20% of features drive 60–80% of cost; targeting those is where the wins are.

Tradeoffs: High-cardinality per-request logging adds observability cost and PII surface; sample or aggregate to control it.

Interview Lesson: The first move in any cost problem is *attribution*, not optimization. You cannot fix what you cannot see per unit.

System design scenario — “Design a cost dashboard for a SaaS AI assistant”

A senior answer covers: (1) **Requirements** — per-feature/user/model cost, daily freshness, anomaly alerts. (2) **Data model** — one immutable event per external call with token fields and a `success` flag. (3) **Pipeline** — middleware → buffered queue → warehouse → rollup job. (4) **Metrics** — cost/request, cost/successful-task, accuracy-per-dollar, tokens/task, cache-hit-rate. (5) **Guardrails** — budget alerts and anomaly detection on day-over-day deltas. (6) **Cost of the dashboard itself** — bounded by sampling high-volume traces. We build this end to end in Chapter 17.

Instrument every external call with a structured cost event before you attempt a single optimization. Attribution converts “the bill is too high” into “feature X on model Y for cohort Z costs \$N per successful task,” which is a solvable engineering problem.

1.6 Optimization playbook and checklist

1. Find the largest line item in the stack (almost always inference).
2. Within inference, split input vs. output vs. cached — attack the largest.
3. Identify the top 3 features by cost per successful task.
4. Check whether a smaller model clears the quality bar for the cheap-and-frequent features.
5. Check whether static prefixes (system prompt, documents, tools) are cacheable.
6. Only then move to micro-optimizations.

Implementation Checklist: Foundations

- ☐ Every external call emits a structured cost event with token counts.
- ☐ Cost is attributable per feature, model, and user cohort.
- ☐ You report cost per *successful* task, not just per request.
- ☐ accuracy-per-dollar is tracked alongside raw cost.
- ☐ Budget alerts fire on anomalous day-over-day deltas.

1.7 Common mistakes

- **Optimizing a total, not a unit.** The total tells you nothing about which feature to fix.
- **Counting requests, not tokens.** Two requests can differ 100× in cost.
- **Ignoring failed/retried work.** Cost per successful task is the honest number.
- **Forgetting the non-inference stack.** Tool, human-review, and vector-DB costs surprise teams.
- **No attribution.** Without per-call events you are flying blind.

1.8 Interview questions

Interview Questions — Fundamentals

Q1. Walk me through the full cost stack of an AI feature.

Answer: Enumerate the eleven line items — inference, embedding, vector DB, reranking, tools/APIs, storage, orchestration, evaluation, observability, human review, infrastructure. Note inference usually dominates for API products and infra for self-hosted, and that tool and human-review costs can exceed inference in agentic or human-in-the-loop systems. Strong candidates immediately ask “what is the workload?” because the dominant term depends on it.

Q2. Why is cost per successful task better than cost per request?

Answer: Cost per request hides retries and failures. A system that retries three times to succeed has 3× the true unit cost. Cost per successful task includes the cost of failed attempts in the numerator and only counts successes in the denominator, so it is the number that maps to gross margin. It also correctly penalizes “cheap but unreliable” designs.

Q3. A monthly bill doubled with flat user count. How do you diagnose it?

Answer: Attribute first. Slice the new events by feature, model, and prompt version. Common causes: a prompt change that grew the system prompt, history that now exceeds a cache threshold, a new feature sending large context, an agent retry loop, or a silent model upgrade. Compare tokens/task and tool-calls/task before vs. after; the metric that moved points to the cause.

Q4. How would you set unit-economics targets for a new AI feature?

Answer: Start from price and target gross margin to get a cost-per-successful-task ceiling. Decompose that ceiling into a token budget and tool-call budget per task, pick a model tier that fits, and design caching/RAG to stay under budget. Treat the ceiling as a design constraint, like a latency SLO.

Q5. When is the non-inference part of the stack the thing to optimize?

Answer: When attribution shows it dominates: search-heavy agents (tool cost), human-in-the-loop workflows (review cost), large always-on vector indexes for low-traffic features (vector-DB cost), or very low-traffic services (orchestration/infra overhead). The discipline is to let attribution, not assumption, choose the target.

Summary Notes. *“Show me the bill divided by successful tasks, sliced by feature and model. That single view tells me where 80% of the savings live before I touch any code.”*